# Lab Activity: Exploring N-grams and Word Probabilities

**Objective:**
how to generate N-grams from a text corpus and calculate simple probabilities of word sequences.

## Step 1: Load Text

```
text = """I love natural language processing. I love learning Python. NLP is fun and powerful."""
text = text.lower().replace(".", "")
words = text.split()
print (words)
```

**Explanation:**

1. `text = """..."""` → Stores your sample text in a variable called `text`.
2. `text.lower()` → Converts all letters to lowercase so `"I"` and `"i"` are treated the same.
3. `.replace(".", "")` → Removes periods so they don't interfere with words.
4. `words = text.split()` → Splits the text into a list of words. Example: `["i", "love", "natural", "language", ...]`.
5. `print(words)` → Shows the list of words so you can check it.

## Step 2: Create N-grams

```
from nltk import ngrams



# Example: Bigram

bigrams = list (ngrams(words, 2))
print("Bigrams:", bigrams)
# Example: Trigram
trigrams = list (ngrams(words, 3))
print ("Trigrams:", trigrams)
```

**Explanation:**

1. from nltk import ngrams → Imports a function that makes N-grams from a list of words.
2. ngrams(words, 2) → Creates bigrams (sequences of 2 words).
3. list() → Converts the result into a list so we can see it.
4. print() → Displays the generated N-grams.
5. ngrams(words, 3) → Creates trigrams (sequences of 3 words).

## Step 3: Count Frequencies

```
from collections import Counter

bigram_counts = Counter(bigrams)
print ("Bigram Frequencies:", bigram_counts)

trigram_counts = Counter(trigrams)
print ("Trigram Frequencies:", trigram_counts)
```

**Explanation:**

1. from collections import Counter → Imports a tool to count how many times each item appears.
2. Counter(bigrams) → Counts how many times each bigram occurs in your text.
3. print() → Shows the counts, e.g., ("i", "love"): 2.
4. Same for trigrams.

## Step 4: Calculate Probabilities

```
bigram_prob = bigram_counts[("i", "love")] / sum (bigram_counts[("i", w)] for w in words if ("i", w) in bigram_counts)
print ("P('love'|'I') =", bigram_prob)
```

**Explanation:**

1. bigram_counts[("i", "love")] → Counts how many times the bigram ("i", "love") appears.
2. sum(...) → Counts how many times "i" appears at the start of any bigram.
3. Division / → Gives the probability of "love" coming **after "i".
4. print() → Shows the probability, e.g., 0.67 means "love" comes after "i" 67% of the time.

## Step 5: Extension Activity

- Generate the most probable next word for a given sequence.